NeuroVision AI: Project Concept Note

Saranya | Marwadi University

15 October 2026

Contents

1 NeuroVision AI — Project Concept Note 1
1.1 1. Executive summary 2
1.2 2. Problem context 2
1.3 3. Aim and objectives 2
1.3.1 Aim 2
1.3.2 Objectives 2
1.4 4. Scope 3
1.4.1 Included in the repository 3
1.4.2 Outside the demonstrated scope 3
1.5 5. Intended users and use case 3
1.6 6. Technical approach 3
1.6.1 Application flow 3
1.6.2 Model design 4
1.6.3 Application stack 4
1.7 7. Dataset and evaluation 4
1.7.1 Evaluation needed for a defensible model claim 5
1.8 8. Responsible use, privacy, and limitations 5
1.9 9. Expected outputs 5
1.10 10. Development roadmap 6
1.11 11. Conclusion 6
1.12 Repository evidence reviewed 6

1 NeuroVision AI — Project Concept Note

Project: NeuroVision AI: Explainable Brain MRI Image Classification Prototype
Student: Saranya (GitHub: `saranya230506`)
Institution: Marwadi University (as recorded in the project configuration)
Project area: Machine Learning and Applied AI
Development context: IBM SkillsBuild / AICTE internship materials are identified in the repository; confirm the exact cohort and submission wording against the student's current program documents before submitting.
SDG alignment: SDG 3, Good Health and Well-Being (educational alignment only)
Project links: GitHub repository · Demonstration website

Project status and safety: NeuroVision AI is an educational software prototype. It is not validated for clinical use and must not be used to diagnose, triage, or make treatment decisions. The current repository contains a prediction fallback that can infer a label from a filename or image heuristic when trained model inference is unavailable. In that path, the displayed heatmap is synthetic, not Grad-CAM. The saved evaluation metrics are reported separately below and have not been independently reproduced from the current deployment.

1.1 1. Executive summary

NeuroVision AI is a web application prototype for demonstrating a brain MRI image-classification workflow. Its intended model task is to assign one of four dataset labels—Glioma, Meningioma, Pituitary, or No Tumor—to a single uploaded image. The application is built with Flask and includes image preprocessing, a TensorFlow/Keras EfficientNetB0 model path, Grad-CAM code, SQLite-backed scan history, a PDF summary generator, and a browser interface.

The project explores how image classification and visual model explanations can be presented together in an accessible demonstration. Its current contribution is an end-to-end software prototype, not a clinical diagnostic service. The model and explanation pipeline need reproducible validation, and the fallback behavior needs to be removed or clearly isolated before any performance claim can be attributed to a live prediction.

1.2 2. Problem context

Brain MRI interpretation requires trained clinical expertise. A student-built image classifier can be a useful way to study preprocessing, transfer learning, model evaluation, and explainability, but a small curated image collection does not establish that a system is safe or effective in a hospital. The project therefore treats the problem as an educational engineering question:

Can a web application demonstrate a complete workflow for classifying four brain MRI image labels and presenting a visual explanation, while making the limits of the model visible to its users?

The current prototype accepts image files rather than a complete DICOM study or volumetric MRI series. It does not perform tumor segmentation, identify tumor grade, or replace radiologist interpretation.

1.3 3. Aim and objectives

1.3.1 Aim

Build and document an educational prototype that demonstrates a four-label brain MRI image-classification pipeline and an associated web interface.

1.3.2 Objectives

1. Organize the image dataset into training and test folders for four labels.
2. Implement an EfficientNetB0 transfer-learning model with a four-class output.
3. Provide image preprocessing and a Grad-CAM implementation for model-based visual inspection.

4. Build a Flask interface for image submission, prediction display, scan history, and PDF summary generation.
5. Record evaluation results and make the evaluation method, data split, and model artifact reproducible.
6. State the system's limitations and prevent demo outputs from being mistaken for clinical advice.

1.4 4. Scope

1.4.1 Included in the repository

- Single-image classification into Glioma, Meningioma, Pituitary, or No Tumor.
- EfficientNetB0 model construction and a two-phase training script.
- Image resizing and contrast preprocessing utilities.
- Grad-CAM code for cases where a trained model is actually used.
- Flask pages and API routes for image uploads and prediction results.
- SQLite persistence for scan records.
- PDF summary generation, history, and dashboard features.

1.4.2 Outside the demonstrated scope

- Clinical diagnosis, triage, or treatment recommendations.
- DICOM ingestion, multi-sequence or volumetric analysis, segmentation, tumor grading, or prognosis.
- Validation across hospitals, MRI scanners, acquisition protocols, age groups, or patient populations.
- PACS/RIS integration, regulatory clearance, security certification, or clinical workflow approval.

1.5 5. Intended users and use case

The intended users are students, educators, and project reviewers examining an applied computer-vision workflow. A safe demonstration uses public or synthetic images, avoids personal health information, and explains the source and limits of each displayed result. The application may be shown as a software demonstration; it is not intended for patients or care providers to make decisions.

1.6 6. Technical approach

1.6.1 Application flow

1. A user selects a supported image file in the web interface.
2. The server loads and preprocesses the image to the configured input dimensions.
3. The predictor attempts to run a loaded TensorFlow model.
4. When model inference succeeds, the application returns the four class scores and can compute Grad-CAM from the configured convolutional layer.
5. The result and optional image artifacts are saved, and the interface can show a history record and PDF summary.

Important implementation qualification: `src/predict.py` invokes `_cv_feature_fallback` when no model is loaded, inference fails, or certain confidence conditions are met. This fallback can choose a class from words in the uploaded filename and otherwise uses a simple image heuristic with hard-coded confidence values. If true model inference is not used, `_generate_saliency_heatmap` makes a heuristic overlay. Such output must not be described as a model prediction or Grad-CAM explanation.

1.6.2 Model design

The model code in `src/model.py` builds an ImageNet-initialized EfficientNetB0 backbone with a custom classification head: global average pooling, batch normalization, a 256-unit dense layer, dropout, and a four-unit softmax layer. Configuration specifies 224 × 224 RGB input, 10 feature-extraction epochs, and 15 fine-tuning epochs. These are implementation settings; they do not by themselves establish model quality.

1.6.3 Application stack

The repository uses Python, TensorFlow/Keras, Flask, OpenCV, SQLite, and ReportLab. The interface is delivered through HTML/CSS/JavaScript templates. The code includes dashboard and history pages, an upload endpoint, and PDF report routes.

1.7 7. Dataset and evaluation

The repository's dataset folders are organized into `Training` and `Testing` directories with four class folders. The project documentation describes a balanced dataset of 5,600 training and 1,600 test images. These counts should be recomputed from the checked-in dataset before final submission, and the dataset source, license, patient-level separation, duplicate handling, and preprocessing leakage should be documented. An image-level folder split alone does not prove that the test set is independent at the patient level.

The checked-in `models/evaluation_metrics.json` reports the following results for 1,600 test images:

Measure	Value recorded in the artifact
Accuracy	0.9638
Macro F1	0.9632
Weighted F1	0.9639
Macro ROC-AUC	0.9880

Class	Precision	Recall	F1	ROC-AUC
Glioma	0.952	0.948	0.950	0.982
Meningioma	0.945	0.951	0.948	0.984
Pituitary	0.981	0.984	0.982	0.995
No Tumor	0.978	0.973	0.975	0.991

These values are transcribed from a JSON file in the repository. They are **reported artifact values**, not independently verified results in this concept note. The checkout inspected for this document did not contain a saved trained-model weights file, and the live prediction code includes a heuristic fallback. The repository therefore does not establish that the currently deployed application produced these metrics using the stated model. The claimed inference time in earlier drafts is omitted because no reproducible timing benchmark was found.

1.7.1 Evaluation needed for a defensible model claim

- Record the dataset source, version, license, and class counts.
- Split data at patient level where patient identifiers are available; check for duplicates and near-duplicates across splits.
- Save the exact training configuration, random seed, software versions, and model checkpoint hash.
- Re-run the test evaluation using the saved checkpoint and test-only data; save predictions, confusion matrix, per-class metrics, and ROC calculations.
- Measure latency on named hardware, with warm-up and repeated runs, and distinguish preprocessing, inference, and end-to-end response time.
- Evaluate Grad-CAM localization separately; a visually plausible overlay is not proof of explanation quality.
- Compare demo behavior with the model path and label fallback or error states in the interface.

1.8 8. Responsible use, privacy, and limitations

- The application is for education and software demonstration only; no output should guide care.
- The current fallback can return filename-derived or heuristic classes. Until removed, it makes user-facing confidence values unreliable when the model path is unavailable.
- The fallback heatmap is not Grad-CAM and must be labeled accordingly or disabled.
- The PDF is an application-generated summary. It is not a radiologist report, certified document, or clinical record.
- The repository stores uploaded images and metadata in local application paths and SQLite. Local storage is not, by itself, evidence of HIPAA, GDPR, or other privacy compliance.
- Do not submit identifiable patient data to the public demonstration site. Use only images that are cleared for demonstration and follow the dataset's license and privacy terms.
- Dataset balance does not establish demographic or scanner fairness. External validation and subgroup analysis have not been documented.
- The system's four labels are dataset categories. "No Tumor" does not mean a scan is clinically normal or exclude other disease.

1.9 9. Expected outputs

1. A reproducible four-class training and evaluation pipeline.
2. A web demonstration that distinguishes genuine model inference from unavailable-model or error states.
3. A documented Grad-CAM path whose output is shown only when computed from the trained model.

4. A project report, Lean Canvas, and presentation that state current evidence and limitations accurately.

1.10 10. Development roadmap

	Stage	Work	Completion evidence
1.	Evidence cleanup	Identify dataset source and licensing; audit splits and duplicates	Dataset card, split manifest, count report
2.	Reproducible model	Train and save the intended checkpoint; freeze evaluation inputs	Checkpoint hash, configuration, predictions, metrics
3.	Predictor safety	Remove filename-driven labels and hard-coded confidence fallback	Tests showing model-unavailable state returns an error, not a diagnosis
4.	Explanation integrity	Disable synthetic heatmaps or label them as non-model visuals	Verified Grad-CAM output linked to model inference
5.	Demo readiness	Use non-identifiable example images; document storage and deployment behavior	Demo checklist and privacy notice
6.	Future research	Consider external validation and DICOM only after earlier stages	Approved protocol and documented evaluation plan

1.11 11. Conclusion

NeuroVision AI is a substantial educational full-stack prototype combining a planned EfficientNetB0 classifier, explainability code, a web application, scan history, and PDF summaries. Its value at this stage is in demonstrating the components of an applied AI workflow and identifying what evidence is needed for a reliable model evaluation. The saved metrics should remain explicitly attributed to the repository artifact until they can be reproduced from a known checkpoint. The current heuristic fallback and generated heatmaps prevent the live application from being presented as a dependable diagnostic system.

1.12 Repository evidence reviewed

- `config.py` — model input size, class names, and training settings.
- `src/model.py` and `src/train.py` — model construction and training workflow.
- `src/predict.py` — inference, fallback, and heatmap behavior.
- `src/gradcam.py` — model-based Grad-CAM implementation.
- `models/evaluation_metrics.json` and `models/training_history.json` — recorded evaluation and training values.

- `app.py`, `database/db.py`, and `src/report_generator.py` — web routes, storage, and generated PDF summary.

Prepared from the supplied repository checkout on 15 October 2026. Verify program metadata and rerun dataset/model evidence checks before formal submission.